

CS3 Code Review - Lab 4 Roster (Version 2)

Name: _____

Date: _____

Choose the best answer for each question. Assume the Lab 4 roster programs are used exactly as provided.

1. What is the minimum valid command line accepted by the `argc < 3` check?

- A. The program name only
- B. The program name and one dropout file
- C. The program name, one course file, and one dropout file
- D. The program name and three course files

2. Why does the course-processing loop use `i < argc - 1`?

- A. The last argument is reserved for the dropout file
- B. Array indexes cannot reach `argc - 1`
- C. The first course is stored at `argv[0]`
- D. It prevents more than one course from being read

3. Suppose a roster file ends with the single word `Taylor` and no last name. What does `readRoster()` do with that word?

- A. Adds a student whose last name is empty
- B. Adds `Taylor Taylor`
- C. Throws an exception
- D. Does not add it because extracting both `firstName` and `lastName` fails

4. What does `courseName("cs4")` return when the filename has no extension?

- A. An empty string
- B. `cs4`
- C. `cs`
- D. The program exits

5. What does `findStudent()` return when no entry has the requested name?

- A. `students.begin()`
- B. A null pointer
- C. `students.end()`
- D. A newly created `StudentEntry`

6. In the procedural version, why is this assignment made after adding a new entry?

```
student = studentEntries.end() - 1;
```

- A. To point student at the newly appended entry
- B. To sort the vector from last to first
- C. To remove the last entry
- D. To move the vector into student

7. Why would it be unsafe to keep using an old vector iterator after studentEntries.push_back(...)?

- A. push_back() always deletes the vector
- B. Vector reallocation may invalidate its iterators
- C. Iterators can refer only to constant elements
- D. push_back() changes every stored student's name

8. What is the purpose of these two statements?

```
auto enrolledCourse = student->begin();  
++enrolledCourse;
```

- A. Move the student's name to the end
- B. Skip the stored name so only course entries are searched
- C. Delete the first course
- D. Advance to the next student in the vector

9. If the same student's name occurs twice in cs2.txt, how many times is cs2 stored in that student's entry?

- A. Zero times
- B. Once
- C. Twice
- D. The entire student entry is removed

10. Before dropouts are removed, what determines the order of studentEntries in the procedural version?

- A. Descending last name
- B. The order of the dropout file
- C. The order in which each distinct student is first encountered across course files
- D. The number of courses each student takes

11. What is the role of `std::remove_if()` in the erase-remove expression?

- A. It destroys the vector itself
- B. It immediately reduces the vector's size
- C. It sorts dropout entries at the back
- D. It moves retained entries forward and returns the new logical end

12. Why is `studentEntries.erase(...)` still needed after `std::remove_if()`?

- A. To physically erase the unwanted tail and reduce the vector's size
- B. To call `readRoster()` again
- C. To restore the original student order
- D. To convert the vector into a list

13. In `[&dropouts](const StudentEntry& student)`, what does `&dropouts` mean?

- A. The lambda receives a pointer to each student
- B. The lambda captures the existing dropouts list by reference
- C. The lambda creates a deep copy of dropouts
- D. The lambda may modify a const student entry

14. In the procedural sorting lambda, what value is compared by `left.front() < right.front()`?

- A. Each student's first course
- B. Each student's last name only
- C. Each student's complete stored name string
- D. The number of items in each entry

15. Why is `unique()` unnecessary in `classSchedule.cpp` after the course files have been processed?

- A. Duplicate names were merged into one student entry during processing
- B. Vectors automatically reject duplicate elements
- C. Dropout removal deletes every duplicate
- D. `std::sort()` also removes duplicates

16. What does `roster.emplace_back(firstName, lastName)` do in the object-oriented `readRoster()`?

- A. Creates a temporary list and splices it into `roster`
- B. Constructs a `Student` directly at the end of the list
- C. Adds two separate string elements to the list
- D. Calls the `Student` move-assignment operator

17. What is copied by the defaulted `Student` copy constructor?

- A. Only `firstName_`
- B. Only the two name strings
- C. The two name strings and the `courses_` list
- D. Nothing, because a defaulted copy constructor is disabled

18. Why is the `Student` move constructor declared `noexcept`?

- A. It guarantees that all file operations succeed
- B. It prevents the moved-from object from being destroyed
- C. It makes the class immutable after a move
- D. It promises the move will not throw and lets containers safely prefer moving

19. Immediately after `std::move(student)` inserts a new student, which statement about the source student is correct?

- A. It remains a valid object, but its moved-from contents should not be relied upon
- B. It must still contain all original strings and courses
- C. It becomes a dangling reference
- D. Accessing its destructor later is undefined behavior

20. How does the object-oriented `operator<` order these students?

```
Alex Young  
Alex Adams  
Beth Adams
```

- A. Alex Young, Alex Adams, Beth Adams
- B. Beth Adams, Alex Adams, Alex Young
- C. Alex Adams, Alex Young, Beth Adams
- D. Alex Adams, Beth Adams, Alex Young

Answer Key

1	C	11	D
2	A	12	A
3	D	13	B
4	B	14	C
5	C	15	A
6	A	16	B
7	B	17	C
8	B	18	D
9	B	19	A
10	C	20	C